Environments

Variable	Where Set	Value	Purpose
`PYTHONPATH`	`>_ run_gui.sh`	`unset`	**Cleared** to avoid ROS 2 path contamination
`CONDA_PREFIX`	`>_ run_gui.sh`	`e`	Points to the active Conda environment
`PATH`	`>_ run_gui.sh`	`$CONDA_PREFIX/bin:$PATH`	Prepends Conda's Python binaries
`LD_LIBRARY_PATH`	`>_ run_gui.sh` & `M↓ HOW_TO_RUN.md`	`$CONDA_PREFIX/lib:...`	Prevents `CXXABI_1.3.15 not found` errors by prioritizing Conda's C++ libs
`DISPLAY`	`>_ run_gui.sh` & `M↓ HOW_TO_RUN.md`	`:1` (default)	Required for Tkinter GUI and Rerun to find the X display

Read by Python Code

Variable	Where Read	Purpose
`DISPLAY`	`lerobot_gui.py:199`	GUI reads `os.environ.get("DISPLAY", ":1")` as the default display entry in the settings form

Optional / User-Settable

Variable	Where Documented	Purpose
`LEROBOT_RERUN_3D`	`M↓ HOW_TO_RUN.md:189`	`export LEROBOT_RERUN_3D=1` (or `0`) to force-enable or force-disable the live 3D URDF pose streaming in Rerun. If not set, it defaults to the `visualize_3d: True` config setting in `config_agilex_nero.py`.

✅ 6 real training datasets — 400 episodes, 359K frames, 5.8 GB

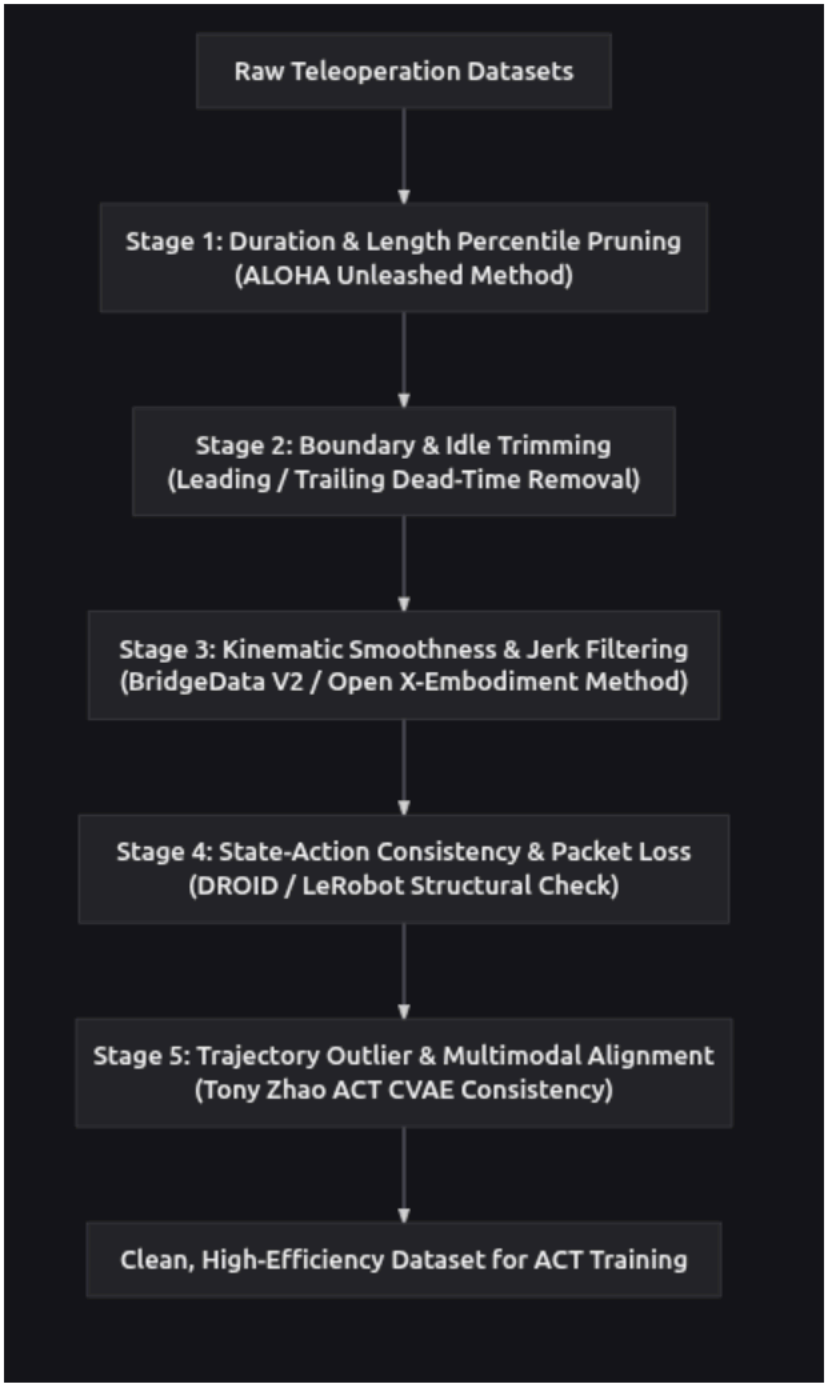
Dataset	Episodes	Notes
`nero_conveyor_sort`	160	Best — most diverse
`nero_sort_80`	80	Supplement
`nero_AJ4Fin`	40	Different operator
`nero_AJTest3`	40	Good quality
`nero_SWAPOCTTest` + `SWAPOCTTest2`	80	Different task (swap)

Next step — per-episode quality check

Once you activate the conda env where `pandas` + `pyarrow` are available, run:

bash

```
conda activate nero-RDP2026Nov
python3 scripts/audit_and_clean_datasets.py --datasets-dir datasets/ --check-episodes
nero_conveyor_sort
```


Raw Teleoperation Datasets
Stage 1: Duration & Length Percentile Pruning
(ALOHA Unleashed Method)
Stage 2: Boundary & Idle Trimming
(Leading / Trailing Dead-Time Removal)
Stage 3: Kinematic Smoothness & Jerk Filtering
(BridgeData V2 / Open X-Embodiment Method)
Stage 4: State-Action Consistency & Packet Loss
(DROID / LeRobot Structural Check)
Stage 5: Trajectory Outlier & Multimodal Alignment
(Tony Zhao ACT CVAE Consistency)
Clean, High-Efficiency Dataset for ACT Training

Originating Method: *BridgeData V2* (Walke et al., CoRL 2023) & *Open X-Embodiment* (Padalkar et al., ICRA 2024).

The Principle

Jerk is the third time-derivative of position (rate of change of acceleration). High jerk indicates:

- Teleoperation controller packet drops or latency hiccups.
- Operator hand tremor or sudden spastic corrections.
- Mechanical fighting or slipping.

Specific Mathematical Formulation

For joint vector $q_t \in \mathbb{R}^7$ sampled at regular intervals Δt:

1. **Discrete Jerk Approximation:**

$$J_t = \frac{\Delta^3 q_t}{\Delta t^3} = \frac{q_{t+2} - 3q_{t+1} + 3q_t - q_{t-1}}{\Delta t^3}$$

2. **Mean Episode Squared Jerk Metric (S_{jerk}):**

$$S_{\text{jerk}}(E) = \frac{1}{T} \sum_{t=1}^{T} \sum_{j=1}^{7} (J_{t,j})^2$$

3. **Filtering Threshold:** Compute the population mean μ_{jerk} and standard deviation σ_{jerk} across all episodes of the task.

$$\text{Flag Episode } E \text{ if } S_{\text{jerk}}(E) > \mu_{\text{jerk}} + 2\sigma_{\text{jerk}} \quad \text{or} \quad S_{\text{jerk}}(E) > P_{95}$$

   Any episode above the 95th percentile is flagged as an erratic trajectory.
4. **Butterworth Signal Smoothing (Optional Preprocessing):** To filter human hand tremor without discarding the trajectory:
   - Apply a **4th-order zero-phase Butterworth low-pass filter** (`scipy.signal.filtfilt`).
   - Cutoff frequency: $f_c = 4.0$–5.0 Hz (for a 30 Hz or 50 Hz recording stream). Human intentional arm movement is < 3 Hz; tremor and CAN packet jit re > 8 Hz.

Our notes

Stage 1: duration and length percentile

$$\text{Reject if } T_i > Q_3 + 1.5 \times \text{IQR}$$

- To get the absolute best AI performance, some researchers are ruthless and delete *everything* that isn't in the fastest 50% or 75% of recordings, ensuring the AI only learns from highly confident, direct movements.

Stage 2: cut out pauses

Specific Algorithm

For each episode with joint positions $q_t \in \mathbb{R}^D$ and sampling period Δt:

1. Calculate the instantaneous L_2 joint velocity:

$$v_t = \frac{\|q_{t+1} - q_t\|_2}{\Delta t}$$

2. Set a stationary threshold ϵ_v (for the 7-DOF Nero arm: $\epsilon_v = 0.02 \text{ rad/s}$).
3. **Leading Trim:** Find the first frame t_{start} where velocity exceeds ϵ_v for at least $k = 5$ consecutive frames:

$$t_{\text{start}} = \min \{t \mid \forall j \in [0, k-1],\, v_{t+j} > \epsilon_v\}$$

- **Measuring Speed (v_t):** The algorithm calculates the physical speed of the robot's joints at every fraction of a second.
- **The Movement Threshold (ϵ_v):** It establishes a baseline speed limit (0.02 rad/s). Any movement slower than this is considered "sitting still" or just sensor static.
- **Leading Trim (Cutting the Start):** The math scans the beginning of the video to find the exact moment the robot moves faster than the threshold for 5 frames in a row. It keeps a tiny fraction of a second right before that moment as a buffer, and deletes all the dead air before it.
- **Trailing Trim (Cutting the End):** It scans the end of the video to find exactly when the task is finished and the robot's speed drops below the threshold for good. It deletes all the dead air at the tail end of the recording.

Stage 3: Kinematic smoothness

Jerk

$$J_t = \frac{\Delta^3 q_t}{\Delta t^3} = \frac{q_{t+2} - 3q_{t+1} + 3q_t - q_{t-1}}{\Delta t^3}$$

Equation:

J_t: The calculated Jerk score at the current time step (t).

q **(Position variables)**: The q variables represent the joint's actual position at specific times.

- q_t is the current position.
- q_{t-1} is the position one frame in the *past*.
- q_{t+1} is the position one frame in the *future*.
- q_{t+2} is the position two frames in the *future*.

The Numerator ($q_{t+2} - 3q_{t+1} + 3q_t - q_{t-1}$): This is a mathematical shortcut called a "third finite difference." To get jerk, you have to measure the change in position (velocity), then the change in velocity (acceleration), and finally the change in acceleration (jerk). The multipliers (the 3s and minus signs) are the direct mathematical result of stacking those three layers of differences into one single formula.

The Denominator (Δt^3): Δt (Delta t) is the set time interval between each recorded frame. Because jerk is the *third* level of change over time (position → velocity → acceleration → jerk), the time gap must be cubed ($\Delta t \times \Delta t \times \Delta t$) to scale the final number correctly.

- Differentiated acceleration (m/s^3)
- Third time derivative of position
- High jerk indicates sudden movements/hand tremor/hiccups etc

Mean jerk

$$S_{\text{jerk}}(E) = \frac{1}{T} \sum_{t=1}^{T} \sum_{j=1}^{7} (J_{t,j})^2$$

- calculates a single score representing the overall "jerkiness" for an entire recorded episode (like one complete task attempt).
- Square ensures positive -> penalise large ones even more -> averages over joints and time steps

Filtering threshold

$$\text{Flag Episode } E \text{ if } S_{\text{jerk}}(E) > \mu_{\text{jerk}} + 2\sigma_{\text{jerk}} \quad \text{or} \quad S_{\text{jerk}}(E) > P_{95}$$

- step explains how to decide if an episode is *too* jerky and should be flagged.
- calculates the average jerkiness (μ_{jerk}) and the variation (σ_{jerk}) across a whole dataset of episodes.
- An episode is flagged as an "erratic trajectory" if its jerk score (S_{jerk}) is significantly higher than average (specifically, if it's more than two standard deviations above the mean, or above the 95th percentile).

Butterworth

- method to clean up the data before analysis, specifically to remove unintentional human hand tremors.
- cutoff frequency is set between 4.0-5.0 Hz because intentional human movement is usually slower (< 3 Hz), while tremors or data errors (jitter) are usually faster (> 8 Hz).
- Limitation: complementary to other methods because it only detects jerkiness but not incomplete recordings

Stage 4: **State-Action Consistency & Hardware Health**

Originating Method: *DROID* (Khazatsky et al., 2024) & *LeRobot Dataset Specification*.

The Specific Checks

1. **Timestamp Monotonicity and Dropped Packets**

$$\Delta t_k = \text{timestamp}_{k+1} - \text{timestamp}_k$$

   - checks for recording lag or missing data.
   - a standard recording rate of 30 frames per second (FPS), a new data frame should arrive every 33.3 milliseconds ($\Delta t_{nominal}$).
   - equation Δt_k calculates the actual time between two frames. If the gap between any two frames is larger than 50 milliseconds ($1.5 \times \Delta t_{nominal}$), it counts as a dropped packet. If an episode has more than 3 of these dropped packets, the data is too stuttery and is thrown out.
2. **Action-State Tracking Divergence** (ensure s that follower followed and did not get stuck or stalled)
   - $$\delta_t = \max_{j \in [1..7]} |a_{t,j} - s_{t,j}|$$
     i. $a_{t,j} - s_{t,j}$**:** It takes the *Action* (a) the human commanded for a specific joint (j) at a specific time (t) and subtracts the actual *State* (s) where the robot's joint actually is.
     ii. $\max_{j \in [1..7]}$ **:** A robotic arm has 7 joints. This tells the system to calculate that error distance for all 7 joints, and then only look at the one joint with the max (biggest) error.
     iii. δ_t **(Lowercase delta** t**):** This final score is the largest single error across all 7 joints at that specific split-second.
   - calculates the difference between the commanded action (a) and actual state (s) across all 7 joints, keeping only the largest error (δ_t).
   - if that error stays above 0.30 radians for more than 10 frames in a row, it means the robot is failing to obey commands eg (it collided with an object, a motor stalled, or the connection dropped)
3. **Task Completion Proxy (Gripper State Delta)** verifies that the robot even opened the hands to attempt task

$$\Delta g = \max_t(g_t) - \min_t(g_t)$$

If $\Delta g < 0.005$ m (5 mm), the gripper never actuated. The robot did not grasp an object. Drop the episode.

- It checks the total range of motion of the robot's gripper (its "hand").
- subtracts the tightest the gripper ever closed ($\min$) from the widest the gripper ever opened ($\max$) during the entire episode.
- 0=fail

4. In manipulation/pick-and-place tasks: Δg=maxt(gt)−mint(gt)\Delta g = \max_{t}(g_t) - \min_{t}(g_t)Δg=tmax(gt)−tmin(gt)

Stage 5: Trajectory consistency and Multimodal Collapse Prevention

Specific Algorithmic Solution: Dynamic Time Warping (DTW) Medoid Filtering

1. Resample all successful episodes in a task to a normalized length $L = 100$ using cubic spline interpolation.
2. For every pair of trajectories (E_i, E_j), compute their pairwise **Dynamic Time Warping (DTW)** distance over the 7 joint angles:

$$D(E_i, E_j) = \text{DTW}\left(q_{1:L}^{(i)}, q_{1:L}^{(j)}\right)$$

3. Identify the **Medoid Trajectory** E^* (the most representative, canonical demonstration):

$$E^* = \arg\min_{E_i} \sum_{j=1}^{N} D(E_i, E_j)$$

4. Calculate the distance of every episode to the canonical medoid: $d_i = D(E_i, E^*)$.
5. **Rejection Rule:**

$$\text{Reject } E_i \text{ if } d_i > \text{Median}(d) + 2.0 \times \text{MAD}(d)$$

(where MAD *is the Median Absolute Deviation).*
This removes outlier trajectories where an operator took an abnormal, convoluted path, leaving only consistent demonstrations that preserve a compact, un-blurred CVAE latent space.

Time warping

- Dynamic Time Warping (DTW) is a mathematical algorithm used to measure the similarity between two sequences (like movements, audio, or video) that happen at different speeds.
    - eg two people saying the word "hello." One says it quickly ("hello"), and the other drags it out ("hellllooooo"). A standard, rigid computer comparison would look at them second-by-second and think they are completely different because the audio waves do not line up perfectly in time.
    - DTW solves this by "warping" or stretching the time axis. It intelligently matches the corresponding parts of the data together—linking the "h" sound to the "h"

sound, and the "o" to the "o"—regardless of how much time passed between them.

1. Resample all successful episodes in a task to a normalized length $L = 100$ using cubic spline interpolation.
2. For every pair of trajectories (E_i, E_j), compute their pairwise **Dynamic Time Warping (DTW)** distance over the 7 joint angles:

$$D(E_i, E_j) = \text{DTW}\left(q_{1:L}^{(i)}, q_{1:L}^{(j)}\right)$$

3. Identify the **Medoid Trajectory** E^* (the most representative, canonical demonstration):

$$E^* = \arg\min_{E_i} \sum_{j=1}^{N} D(E_i, E_j)$$

4. Calculate the distance of every episode to the canonical medoid: $d_i = D(E_i, E^*)$.
5. **Rejection Rule:**

$$\text{Reject } E_i \text{ if } d_i > \text{Median}(d) + 2.0 \times \text{MAD}(d)$$

-
  - **Variables:** $q^{(i)}$ and $q^{(j)}$ represent the two different robot recordings (like Person A and Person B drawing the "S"). The $1{:}\,L$ means it is looking at the full timeline of both recordings, from frame 1 to frame L (100)
  - The algorithm creates a 2D mathematical grid. The bottom axis is the timeline of Episode i (1 to 100 frames), and the side axis is the timeline of Episode j (1 to 100 frames)
  - **Warping (Finding the Path):** The algorithm must draw a line through this grid from the bottom-left (start of the video) to the top-right (end of the video).
  - **The Final Score (D):** The algorithm tests all possible paths through the grid to find the "path of least resistance"—the route that adds up to the lowest total difference between the two poses. That lowest possible total score becomes D.
    - A rigid, second-by-second comparison would force this line to be perfectly diagonal.
    - DTW allows the line to bend. It can step horizontally (advancing time in Episode i while holding Episode j still) or vertically (advancing Episode j while holding i still). This bending is the actual "time warp."

Medoid trajectory: most standard demonstration

$$E^* = \arg\min_{E_i} \sum_{j=1}^{N} D(E_i, E_j)$$

- E^* **(E-star):** The final answer. This represents the "Medoid Trajectory," which is the single most standard, textbook demonstration of the task.

- $D(E_i, E_j)$**:** The Dynamic Time Warping (DTW) distance (difference in physical path) between one specific candidate episode (E_i) and another episode (E_j).
- $\sum_{j=1}^{N}$ **(Summation):** The Greek letter Sigma (Σ) means "add everything up." It calculates the distance from the candidate episode (E_i) to episode 1, then to episode 2, all the way up to the final episode (N), and adds all those distances together into one massive total score.
- the algorithm takes one recording and calculates its DTW distance to every other recording, adding them up to get a total distance score. It repeats this for every single episode. The episode with the absolute lowest total score (the `arg min`) is crowned the Medoid. It sits mathematically perfectly in the center of the group.
- Medoid: the most centrally located data point in a cluster

The Rejection Rule

- statistical bouncer that throws away weird, wandering, or convoluted human demonstrations so they do not confuse the AI during training.

$$\text{Reject } E_i \text{ if } d_i > \text{Median}(d) + 2.0 \times \text{MAD}(d)$$

- MAD = Median Absolute Deviation
- It calculates the most centrally located data point in a clusterlates the distance (did_idi) of every individual episode to the Medoid.
- It finds the middle-ground distance across the whole dataset
- If a specific episode's distance is greater than the median plus twice the typical variation, it proves the human operator took a path that was too extreme or unusual. That episode is rejected and deleted.

training an AI model (like ACT) to control a robot, it learns by compressing human demonstrations into a mathematical "latent space" (using a Conditional Variational Autoencoder, or CVAE) and then decoding them back into actions.